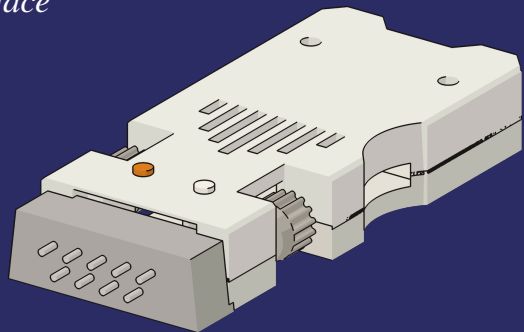


FullNet

*Personal Computer
Hardware Reference
Library*

Technical Reference

The INT F5 Programming Interface



FN-RS232-TR-001

First Edition (2026)

This publication describes the **INT F5** programming interface of the RS-232 FujiNet under MS-DOS: the software interrupt installed by the FujiNet device driver through which any program may command any FujiNet device, with or without a data payload, in either direction.

It is written for assembly-language and C programmers, and for anyone who wishes to understand what the FujiNet C library (**fujinet-lib**) ultimately calls. Every register, command, and return code is reproduced from the canonical sources — the FujiNet device driver, the FujiNet firmware, and the C bindings — so that what is printed here matches the machine exactly.

FujiNet is free and open. The driver, the firmware, the C library, and this manual are released under free-software and open-hardware licenses. Sources for everything live at github.com/FujiNetWIFI.

© 2026 The FujiNet Community · Copy freely

The FujiNet project is a community of enthusiasts and is not affiliated with, endorsed by, or sponsored by International Business Machines Corporation. The visual styling of this manual is an affectionate tribute to the IBM Personal Computer *Technical Reference*.

Preface

The FujiNet PC interface reserves software interrupt **F5h** to provide a single, uniform way to talk to the FujiNet — no matter how it is physically attached. Through one interrupt you can send *any* command the FujiNet understands, to *any* of its virtual devices, carrying a payload to the FujiNet, from the FujiNet, or neither.

This reference has six sections:

- **Section 1. Introduction** describes the FujiNet device model and how INT F5 fits between your program, the device driver, and the FujiNet hardware.
- **Section 2. The Calling Convention** documents the registers, the field descriptor, the return codes, and the wire protocol the driver speaks on your behalf.
- **Section 3. The Devices** lists the device IDs and what each one is for.
- **Section 4. Command Reference** documents every command — its direction, parameters, payload, and return — device by device, with an example for each group.
- **Section 5. A Worked Example** builds a complete network utility, **NETCAT**, in assembly language and again in C.
- **Section 6. A Second Example** reads a Mastodon timeline through the FujiNet's JSON engine, in assembly, C, and Pascal side by side.
- **Section 7. Inside FUJINET.SYS** is a guided tour of the block device driver — the program that installs INT F5 and presents the FujiNet's disks as DOS drive letters — with source listings.
- **Section 8. Inside FUJIPRN.SYS** does the same for the printer driver, including its INT 17h hook and idle-time auto-flush.
- **Section 9. Appendices** gather the error codes, the status request types, the field-descriptor table, the wire-frame layout, and a one-line summary of every command.

Note: Throughout this reference, register and command values are given in hexadecimal with a trailing **h** (for example **70h**) or with a leading **0x** in C and assembler listings, as the source uses both.

Contents

Section 1. Introduction	1-1
The Device Model · The Software Layers · Relation to fujinet-lib	
Section 2. The Calling Convention	2-1
Registers · Payload Direction · The Field Descriptor · Return Codes · The C Wrappers · The Wire Protocol	
Section 3. The Devices	3-1
Device ID Map · Device Classes	
Section 4. Command Reference	4-1
The Control Device · Network Adapters · Disk Drives · The Clock	
Section 5. A Worked Example: NETCAT	5-1
What It Does · Opening the Connection · The Polling Loop · Reading and Writing · The Whole Program · The Same Program in C	
Section 6. A Second Example: Reading Mastodon	6-1
The JSON Engine · The Flow · In C · In Assembler · In Pascal	
Section 7. Inside FUJINET.SYS	7-1
The Device Header · Strategy and Interrupt · The Request Packet · The Dispatch Table · Initialization · Reading and Writing Sectors · Media Check and BPB · The IOCTL Signature	
Section 8. Inside FUJIPRN.SYS	8-1
A Character Device · Capturing Output · The INT 17h Hook · The Buffer · Auto-Flush: Timer and Idle	
Section 9. Appendices	9-1
Network Error Codes · Status Request Types · Field Descriptors · The FujiBus Frame	

SECTION 1. INTRODUCTION

The FujiNet presents itself to the Personal Computer not as one device but as a small bus of *virtual devices*: eight disk drives, eight network adapters, a real-time clock, a printer, and a control device that manages them all. The FujiNet device driver — **FUJINET.SYS**, loaded from **CONFIG.SYS** — installs a single software interrupt, **INT F5h**, that reaches every one of them.

A program issues a command by loading a handful of registers and executing **INT F5h**. The driver packages the command, sends it to the FujiNet across the serial cable, waits for the reply, copies any returned data into your buffer, and hands control back with a one-character result in **AL**. The program need not know anything about serial ports, baud rates, framing, or checksums; the driver does all of it.

The Device Model

Every FujiNet command is addressed to a **device** (a one-byte ID in **AL**) and names a **command** (a one-byte code in **AH**). The same command byte may mean different things to different devices — '**R**' is "read a sector" to a disk and "read bytes" to a network adapter — so the device ID always travels with it. Section 3 lists the devices; Section 4 documents their commands.

The Software Layers

Four layers sit between your program and the network:

- **Your program** loads registers and calls **INT F5h**.
- **The INT F5 handler** in **FUJINET.SYS** translates the registers into a FujiNet bus command.
- **The FUJICOM layer** frames that command as a packet, wraps it in SLIP, and exchanges it with the FujiNet over the serial port (Section 2, *The Wire Protocol*).
- **The FujiNet** carries out the command — touching a disk image, a network socket, the clock — and replies.

You work entirely at the top layer. Everything below **INT F5h** is the driver's concern, and is described here only so you understand what your calls set in motion.

Relation to fujinet-lib

The FujiNet C library, **fujinet-lib**, provides comfortable C functions — **network_open**, **fuji_mount_image**, and the rest. On MS-DOS, every one of those functions ultimately reduces to an **INT F5h** call through one of three thin wrappers: **int_f5**, **int_f5_read**, and **int_f5_write** (Section 2). This reference documents that floor — the exact interface the library stands on — so you may call it directly from assembler or C, or understand precisely what the library does on your behalf.

SECTION 2. THE CALLING CONVENTION

To issue a command, load the registers below and execute **INT F5h**. On return, **AL** holds the result.

Registers

DX DH — field descriptor

DL — payload direction

AX AH — command

AL — device ID

CX CH — AUX2

CL — AUX1

SI AUX3 (low byte) : AUX4 (high byte)

ES:BX segment : offset of the payload buffer

DI length of the payload buffer, in bytes

Register	Holds
DL	Payload direction: 00h none, 40h read, 80h write
DH	Field descriptor: how many AUX bytes to send (see below)
AL	Device ID (Section 3)
AH	Command code (Section 4)
CL	AUX1 — first command parameter
CH	AUX2 — second command parameter
SI	AUX3 and AUX4 — third and fourth parameters (low : high)
ES:BX	Far pointer to the payload buffer (read or write)
DI	Length of the payload buffer, in bytes

Note: **ES:BX** and **DI** are used only when **DL** is **40h** or **80h**. For a no-payload command (**DL=00h**) they are ignored.

Payload Direction

Register **DL** selects one of three forms:

DL	Form	Meaning
00h	None	Send the command alone; no data moves.
40h	Read	The FujiNet returns data into ES:BX, DI bytes.
80h	Write	Your data at ES:BX, DI bytes, goes to the FujiNet.

The Field Descriptor

A command's AUX bytes only travel to the FujiNet if the **field descriptor** in **DH** says how many to send. Set **DH** to match the parameters the command expects:

Name	DH	Bytes	Sends
FUJI_FIELD_NONE	00h	0	no AUX
FUJI_FIELD_A1	01h	1	AUX1
FUJI_FIELD_A1_A2	02h	2	AUX1, AUX2
FUJI_FIELD_A1_A2_A3	03h	3	AUX1, AUX2, AUX3
FUJI_FIELD_A1_A2_A3_A4	04h	4	AUX1–AUX4
FUJI_FIELD_B12	05h	2	AUX1, AUX2 (16-bit field)
FUJI_FIELD_B12_B34	06h	4	two 16-bit fields
FUJI_FIELD_C1234	07h	4	AUX1–AUX4 as one 32-bit field

The AUX bytes are placed into the command frame in order — AUX1, then AUX2, and so on — up to the count the descriptor names. A command that takes a 16-bit length passes it as AUX1 (low) and AUX2 (high) with **FUJI_FIELD_A1_A2** a disk sector number is a 32-bit value passed in AUX1–AUX4 with **FUJI_FIELD_C1234**.

Warning: A command whose AUX bytes you set in **CL/CH** but whose field descriptor in **DH** is left at **00h** will reach the FujiNet with *no* parameters, and will usually be rejected. Always set **DH** to match the command.

Return Codes

On return, **AL** holds a single character:

AL	Name	Meaning
'C'	Complete	The command finished without error.
'E'	Error	The command finished, but a problem occurred.
'N'	NAK	The device did not recognize the command.

For read commands, a result of 'C' means your buffer now holds the returned data. For many network commands the *meaning* of an error is found by a following STATUS call, whose fourth byte is a detailed error code (Appendix A).

The C Wrappers

The FujiNet C library exposes the interface as three functions — one per direction. They are the whole of the library's floor; everything else is built on them.

```
unsigned char int_f5(unsigned char dev, unsigned char cmd, unsigned char aux1,
unsigned char aux2); unsigned char int_f5_read(unsigned char dev, unsigned char
cmd, unsigned char aux1, unsigned char aux2, void buf, unsigned short len); unsigned
char int_f5_write(unsigned char dev, unsigned char cmd, unsigned char aux1, unsigned
char aux2, void buf, unsigned short len);
```

Each loads **DL** with the direction, **AL** with **dev**, **AH** with **cmd**, **CL/CH** with the AUX bytes, and (for read and write) **ES:BX/DI** with the buffer, then executes **INT F5h** and returns **AL**. They set **DH** and **SI** to zero; when a command needs AUX bytes or AUX3/AUX4, set **DH** (and **SI**) yourself, as the assembler examples in Section 4 show.

The simplest possible call — reset the FujiNet — needs no payload and no parameters:

```
MOV DL,00h      ; direction: none
MOV DH,00h      ; field descriptor: none
MOV AL,70h      ; device: FujiNet control
MOV AH,0FFh     ; command: RESET
INT 0F5h        ; AL = 'C' on success
```

The Wire Protocol

You do not need this section to use INT F5, but it explains what the driver builds. Each command becomes a **FujiBus packet**: a five-byte header (device, command, 16-bit length, 8-bit checksum, field descriptor), followed by the AUX bytes the descriptor names, followed by any payload. The packet is wrapped in **SLIP** framing — bracketed by **C0h** end markers, with **C0h** and **DB h** escaped — and sent over the serial line. The FujiNet replies with a header whose command byte is **ACK (06h)** on success, followed by any requested data. The driver verifies the checksum and the device, then reports 'C' or 'E'.

SECTION 3. THE DEVICES

Register **AL** names the destination device. The FujiNet recognizes the following IDs:

AL	Device
31h – 38h	Disk drives 1 through 8 (block devices)
40h – 43h	Printer (driven by FUJIPRN.SYS see note)
45h	Real-time clock (NTP)
70h	FujiNet control device
71h – 78h	Network adapters 1 through 8 (character devices)

Note: The printer device (**40h**) is served by the printer driver, **FUJIPRN.SYS**, which hooks **INT 17h** printing is done with ordinary DOS printer output rather than through **INT F5**, and is not covered here. Devices for serial, CP/M, MIDI, and voice exist in the firmware but are not used by the MS-DOS driver.

Device Classes

- **Disk drives (31h-38h)** are block devices. They read and write 512-byte sectors of the disk image mounted in their slot. The MS-DOS driver exposes these to DOS as drive letters; you may also reach them directly (Section 4.3).
- **The clock (45h)** returns the current date and time, kept by the FujiNet from the internet (Section 4.4).
- **The control device (70h)** manages the FujiNet itself: WiFi, hosts, disk slots, directories, the adapter configuration, and more. It is the largest command set (Section 4.1).
- **Network adapters (71h-78h)** are character devices. Each is an independent connection — to a web server, a TCP socket, a file — opened with a URL and read or written as a stream (Section 4.2).

SECTION 4. COMMAND REFERENCE

This section documents the commands of each device. Every entry names the command's direction, its field descriptor, the meaning of its AUX bytes and payload, and what it returns, followed by a short description. An example accompanies each group.

4.1 The Control Device (70h)

The control device manages the FujiNet. Its commands fall into groups: wireless networking, host and disk-slot management, directory browsing, adapter information, and housekeeping. The most-used commands are detailed below; every command is summarized in the table at the end of this section.

RESET — Restart the FujiNet

Dev 70h Cmd FFh

Direction	None
Field (DH)	NONE
AUX	—
Payload	—
Returns	'C'

Restarts the FujiNet's firmware. The serial link is re-established automatically. Equivalent to pressing the Reset button on the adapter.

MOV	DL,00h	; none
MOV	AX,0FF70h	; AH=FFh RESET, AL=70h FujiNet
INT	0F5h	

GET ADAPTER CONFIG — Read network configuration

Dev 70h Cmd EBh

Direction	Read
Field (DH)	NONE
AUX	—
Payload	139-byte AdapterConf ig in
Returns	'C'

Returns the FujiNet's current network details — SSID, host name, IP address, gateway, netmask, DNS, MAC, BSSID, and firmware version — as one packed structure.

```
typedef struct { char ssid[33]; unsigned char gateway[4]; char hostname[64]; unsigned
char netmask[4]; unsigned char localIP[4]; unsigned char dnsIP[4]; unsigned char
macAddress[6]; unsigned char bssid[6]; char fn_version[15]; } AdapterConfig;
AdapterConfig ac; int_f5_read(0x70, 0xEB, 0, 0, &ac, sizeof(ac));
```

SCAN NETWORKS — Begin a WiFi scan

Dev 70h Cmd FDh

Direction	Read
Field (DH)	NONE
AUX	—
Payload	1 byte: network count, in
Returns	'C'

Scans for wireless networks and returns the number found. Follow with GET SCAN RESULT once per network to read each name and signal strength.

```
unsigned char count;
int_f5_read(0x70, 0xFD, 0, 0, &count, 1);
```

GET SCAN RESULT — Read one scanned network

Dev 70h Cmd FCh

Direction	Read
Field (DH)	A1
AUX	AUX1 = network index
Payload	SSIDInfo in
Returns	'C'

Returns the name and signal strength of the network at the given index (0-based, up to the count from SCAN NETWORKS). Because this command takes a parameter, set the field descriptor in DH.

```
MOV DL,40h      ; read
MOV DH,01h      ; FUJI_FIELD_A1
MOV AX,0FC70h   ; AH=FCh, AL=70h
MOV CL,[index]  ; AUX1 = which network
LES BX,[ssidinfo] ; ES:BX -> buffer
MOV DI,34       ; sizeof SSIDInfo
INT 0F5h
```

SET SSID — Join a wireless network

Dev 70h Cmd FBh

Direction	Write
Field (DH)	NONE
AUX	—
Payload	ssid[32]+password[64] out
Returns	'C'

Sends a network name and password; the FujiNet joins that network and remembers it. The payload is two fixed fields.

```
struct { char ssid[32]; char password[64]; } s; strcpy(s.ssid, "HOMEBASE"); strcpy(s.password, "secret"); int_f5_write(0x70, 0xFB, 0, 0, &s, sizeof(s));
```

GET WIFI STATUS — Read the WiFi link state

Dev 70h Cmd FAh

Direction	Read
Field (DH)	NONE
AUX	—
Payload	1 byte: status, in
Returns	'C'

Returns a single byte: 3 means connected; other values mean the link is down or connecting.

```
unsigned char wifi;  
int_f5_read(0x70, 0xFA, 0, 0, &wifi, 1);
```

MOUNT HOST — Open a host slot

Dev 70h Cmd F9h

Direction	None
Field (DH)	A1
AUX	AUX1 = host slot (0–7)
Payload	—
Returns	'C'

Connects the FujiNet to the file server named in the given host slot, so its disk images can be mounted. Set DH=01h.

```
MOV DX,0100h    ; DH=01 (A1), DL=00 (none)  
MOV AX,0F970h   ; AH=F9h, AL=70h  
MOV CL,01h      ; AUX1 = host slot 1  
INT 0F5h
```

MOUNT IMAGE — Mount a disk image into a drive

Dev 70h Cmd F8h

Direction	None
Field (DH)	A1_A2
AUX	AUX1 = device slot (0–7), AUX2 = mode (1=R/O, 2=R/W)
Payload	—
Returns	'C'

Mounts the disk image previously selected for the given device slot, read-only or read/write. The image and host are chosen with SET DEVICE FULL PATH or by the slot table. Set DH=02h.

```
MOV DX,0200h      ; DH=02 (A1_A2), DL=00
MOV AX,0F870h     ; AH=F8h, AL=70h
MOV CL,00h        ; AUX1 = device slot 0
MOV CH,01h        ; AUX2 = read-only
INT 0F5h
```

UNMOUNT IMAGE — Eject a disk image

Dev 70h Cmd E9h

Direction	None
Field (DH)	A1
AUX	AUX1 = device slot (0–7)
Payload	—
Returns	'C'

Ejects whatever image is mounted in the given device slot.

READ DEVICE SLOTS — Read the disk-slot table

Dev 70h Cmd F2h

Direction	Read
Field (DH)	NONE
AUX	—
Payload	8 × DeviceSlot in
Returns	'C'

Returns the table of eight device (drive) slots: for each, the host it came from, its mode, and the image's file name.

```
typedef struct { unsigned char hostSlot; unsigned char mode; char file[36]; }
DeviceSlot;
DeviceSlot slots[8]; int_f5_read(0x70, 0xF2, 0, 0, slots, sizeof(slots));
```

READ HOST SLOTS — Read the host-slot table

Dev 70h Cmd F4h

Direction	Read
Field (DH)	NONE
AUX	—
Payload	8 × 32-byte names in
Returns	'C'

Returns the eight host-slot names (32 bytes each), the file servers and the SD card the FujiNet knows about.

```
char hosts[8][32];
int_f5_read(0x70, 0xF4, 0, 0, hosts, sizeof(hosts));
```

WRITE HOST SLOTS — Write the host-slot table

Dev 70h Cmd F3h

Direction	Write
Field (DH)	NONE
AUX	—
Payload	8 × 32-byte names out
Returns	'C'

Replaces all eight host-slot names. Read them first, change what you need, and write them back.

WRITE DEVICE SLOTS — Write the disk-slot table

Dev 70h Cmd F1h

Direction	Write
Field (DH)	NONE
AUX	—
Payload	8 × DeviceSlot out
Returns	'C'

Replaces the device-slot table. Used to assign images to slots before mounting.

MOUNT ALL — Mount every selected slot

Dev 70h Cmd D7h

Direction	None
Field (DH)	NONE
AUX	—
Payload	—
Returns	'C'

Mounts every device slot that has an image selected — the operation CONFIG performs as it exits. After this, the disks are live at their drive letters.

OPEN DIRECTORY — Begin reading a host's directory

Dev 70h Cmd F7h

Direction	Write
Field (DH)	A1
AUX	AUX1 = host slot (0–7)
Payload	path string out
Returns	'C'

Opens a directory on a mounted host for reading. The payload is the path to open (for example `MSDOS\`). Follow with READ DIR ENTRY.

READ DIR ENTRY — Read one directory entry

Dev 70h Cmd F6h

Direction	Read
Field (DH)	A1_A2
AUX	AUX1 = max length, AUX2 = options
Payload	entry string in
Returns	'C'

Returns the next directory entry as a string of at most AUX1 bytes. A name ending in `\` is a folder. An entry of 7Fh at the first byte marks the end of the directory.

CLOSE DIRECTORY — Finish reading a directory

Dev 70h Cmd F5h

Direction	None
Field (DH)	NONE
AUX	—
Payload	—
Returns	'C'

Closes the directory opened by OPEN DIRECTORY.

NEW DISK — Create a blank disk image

Dev 70h Cmd E7h

Direction	Write
Field (DH)	NONE
AUX	—
Payload	NewDisk structure out
Returns	'C'

Creates a new, empty disk image on a host. The structure gives the host and device slots, the geometry (sector count and size), and the file name.

```
typedef struct { unsigned short numSectors; unsigned short sectorSize; unsigned char
hostSlot; unsigned char deviceSlot; char filename[256]; } NewDisk;
NewDisk nd = { 720, 512, 0, 0, "" }; strcpy(nd.filename, "BLANK.IMG");
int_f5_write(0x70, 0xE7, 0, 0, &nd, sizeof(nd));
```

All Control-Device Commands

The complete command set of device 70h. Commands shown earlier in full are marked *.

Cmd	Name	Purpose
00h	DEVICE READY	Test that the device responds
BBh	GENERATE GUID	Return a fresh globally-unique identifier
BCh	QRCODE INPUT	Feed text to the QR-code encoder
BDh	QRCODE ENCODE	Encode the input as a QR code
BEh	QRCODE LENGTH	Length of the encoded QR code
BFh	QRCODE OUTPUT	Read the encoded QR code
C1h	GET HEAP	Free memory in the FujiNet
C2h	HASH CLEAR	Reset the hash engine
C3h	HASH COMPUTE	Compute hash, keep input
C4h	GET ADAPTER CFG EXT	Extended adapter configuration
C5h	HASH OUTPUT	Read the computed hash
C6h	HASH LENGTH	Length of the hash output
C7h	HASH COMPUTE	Compute hash and clear
C8h	HASH INPUT	Feed bytes to the hash engine
D2h	GET TIME	Time as a packed value
D3h	RANDOM NUMBER	A hardware random number
D6h	SET BOOT MODE	Choose the boot configuration
D7h	MOUNT ALL ■	Mount every selected slot
D8h	COPY FILE	Copy a file between hosts
D9h	CONFIG BOOT	Enable or disable the CONFIG boot disk
DAh	GET DEVICE FULLPATH	Read a slot's full image path
DBh	CLOSE APPKEY	Close the open app-key
DCh	OPEN APPKEY	Open an app-key for read or write
DDh	READ APPKEY	Read application-private data
DEh	WRITE APPKEY	Write application-private data
E0h	GET HOST PREFIX	Read a host's path prefix

Cmd	Name	Purpose
E1h	SET HOST PREFIX	Set a host's path prefix
E2h	SET DEVICE FULLPATH	Select a slot's image by path
E5h	GET DIR POSITION	Current directory read position
E4h	SET DIR POSITION	Seek within a directory
E6h	UNMOUNT HOST	Disconnect a host slot
E7h	NEW DISK ■	Create a blank disk image
E8h	GET ADAPTER CONFIG ■	Network configuration
E9h	UNMOUNT IMAGE ■	Eject a disk image
EAh	GET WIFI ENABLED	Is the radio switched on
EBh	SET BAUDRATE	Change the serial speed
F1h	WRITE DEVICE SLOTS ■	Write the disk-slot table
F2h	READ DEVICE SLOTS ■	Read the disk-slot table
F3h	WRITE HOST SLOTS ■	Write the host-slot table
F4h	READ HOST SLOTS ■	Read the host-slot table
F5h	CLOSE DIRECTORY ■	Finish reading a directory
F6h	READ DIR ENTRY ■	Read one directory entry
F7h	OPEN DIRECTORY ■	Begin reading a directory
F8h	MOUNT IMAGE ■	Mount a disk image
F9h	MOUNT HOST ■	Open a host slot
FAh	GET WIFI STATUS ■	Read the WiFi link state
FBh	SET SSID ■	Join a wireless network
FCh	GET SCAN RESULT ■	Read one scanned network
FDh	SCAN NETWORKS ■	Begin a WiFi scan
FEh	GET SSID	Read the joined network's name
FFh	RESET ■	Restart the FujiNet

4.2 Network Adapters (71h-78h)

A network adapter is a single connection — to a web server, a TCP socket, a remote file. You OPEN it with a URL, then READ and WRITE it as a stream, and CLOSE it when done. Eight adapters (**71h–78h**) run independently. The URL — the FujiNet calls it a *devicespec* — names the protocol, as in **N:TCP://host:1234/** or **N:HTTP://example.com/page** .

OPEN — Open a network connection

Dev 71h–78h Cmd 4Fh "O"

Direction	Write
Field (DH)	A1_A2
AUX	AUX1 = mode, AUX2 = translation
Payload	devicespec URL out
Returns	'C'/'E'

Opens the connection named by the devicespec. AUX1 is the access mode: **04h** read, **08h** write, **0Ch** read/write. AUX2 selects end-of-line translation: **00h** none, **01h** CR, **02h** LF, **03h** CR/LF. The payload is the URL (pad or terminate to a known length).

```
char url[256] = "N:TCP://192.168.1.10:9000/";
/* mode 0x0C = read/write, trans 0 = none */
/* set DH = FUJI_FIELD_A1_A2 (02h), CL=0x0C, CH=00 */
MOV DX,0280h      ; DH=02 (A1_A2), DL=80 (write)
MOV AX,4F71h      ; AH=4Fh 'O', AL=71h adapter 1
MOV CL,0Ch        ; AUX1 = read/write
MOV CH,00h        ; AUX2 = no translation
LES BX,[url]       ; ES:BX -> devicespec
MOV DI,256         ; length
INT 0F5h
```

STATUS — Bytes waiting and link state

Dev 71h–78h Cmd 53h "S"

Direction	Read
Field (DH)	NONE
AUX	—
Payload	4 bytes in
Returns	'C'

Returns four bytes: the number of bytes waiting to be read (low, then high), a **connected** flag, and an **error** code (Appendix A). Always STATUS before READ, so you ask for no more than is waiting.

```
unsigned char st[4]; int_f5_read(0x71, 'S', 0, 0, st, 4); unsigned short waiting =
st[0] | (st[1] << 8); unsigned char connected = st[2], error = st[3];
```

READ — Read bytes from the connection

Dev 71h-78h Cmd 52h "R"

Direction	Read
Field (DH)	A1_A2
AUX	AUX1/AUX2 = byte count (low/high)
Payload	data in
Returns	'C'/'E'

Reads up to a 16-bit count of bytes into your buffer. Use the count from STATUS. Set DH=A1_A2 and DI to the same count.

```
MOV DX,0240h      ; DH=02 (A1_A2), DL=40 (read)
MOV AX,5271h      ; AH=52h 'R', AL=71h
MOV CX,[waiting]  ; CL/CH = byte count
LES BX,[buf]
MOV DI,[waiting]
INT 0F5h
```

WRITE — Write bytes to the connection

Dev 71h-78h Cmd 57h "W"

Direction	Write
Field (DH)	A1_A2
AUX	AUX1/AUX2 = byte count (low/high)
Payload	data out
Returns	'C'/'E'

Writes a 16-bit count of bytes from your buffer to the connection.

```
network_write equivalent: CL/CH = len (low/high), DH = A1_A2, DL = 80h, AH = 'W' (57h),
AL = device, ES:BX = buf, DI = len.
```

CLOSE — Close the connection

Dev 71h-78h Cmd 43h "C"

Direction	None
Field (DH)	NONE
AUX	—
Payload	—
Returns	'C'

Closes the adapter and frees it for reuse. Always close what you open.

PARSE — Parse received data as JSON**Dev 71h–78h Cmd 50h "P"**

Direction	None
Field (DH)	NONE
AUX	—
Payload	—
Returns	'C'/'E'

Parses the data most recently read on this adapter as JSON, readying it for QUERY. Used with web services that answer in JSON.

QUERY — Select a value from parsed JSON**Dev 71h–78h Cmd 51h "Q"**

Direction	Write
Field (DH)	NONE
AUX	—
Payload	query_string out
Returns	'C'/'E'

Sets a query (a JSONPath-like path) against the parsed JSON. A following STATUS reports the size of the result, which a READ then returns. Lets a program pull one field out of a web reply without a JSON parser of its own.

Other Network Commands

Cmd	Name	Purpose
FCh	CHANNEL MODE	Switch between stream (0) and JSON (1) modes; mode in AUX1
25h	SEEK	Move the read/write position (AUX = 32-bit offset)
26h	TELL	Report the current position
30h	GET PREFIX	Read the adapter's path prefix
2Ch	SET PREFIX	Set the adapter's path prefix
20h	RENAME	Rename a file (file protocols)
21h	DELETE	Delete a file
2Ah	MKDIR	Make a directory
2Bh	RMDIR	Remove a directory
FDh	USERNAME	Supply a login name for the next OPEN
FEh	PASSWORD	Supply a password for the next OPEN
5Ah	SET INT RATE	Set the byte-waiting interrupt rate
41h	CONTROL	Protocol control (TCP accept, etc.)
44h	SET DESTINATION	Set the UDP destination

Cmd	Name	Purpose
72h	GET REMOTE	Read the UDP sender's address

4.3 Disk Drives (31h-38h)

A disk drive reads and writes 512-byte sectors of the disk image in its slot. The MS-DOS driver uses these commands to present each drive as a DOS drive letter; you may also call them directly to read or write raw sectors. The sector number is a 32-bit value passed in AUX1–AUX4 with the **C1234** field descriptor.

READ — Read a 512-byte sector

Dev 31h–38h Cmd 52h "R"

Direction	Read
Field (DH)	C1234
AUX	AUX1–AUX4 = 32-bit sector
Payload	512 bytes in
Returns	'C'/'E'

Reads one sector of the mounted image into your buffer.

```
MOV DX,0740h      ; DH=07 (C1234), DL=40 (read)
MOV AX,5231h      ; AH=52h 'R', AL=31h drive 1
MOV CL,[sec+0]    ; AUX1 sector byte 0
MOV CH,[sec+1]    ; AUX2 sector byte 1
MOV SI,[sec+2]    ; AUX3:AUX4 sector bytes 2,3
LES BX,[buf]
MOV DI,512
INT 0F5h
```

WRITE — Write a 512-byte sector

Dev 31h–38h Cmd 57h "W"

Direction	Write
Field (DH)	C1234
AUX	AUX1–AUX4 = 32-bit sector
Payload	512 bytes out
Returns	'C'/'E'

Writes one sector to the mounted image. The image must be mounted read/write, or the command returns 'E'.

STATUS — Read drive status

Dev 31h–38h Cmd 53h "S"

Direction	Read
Field (DH)	NONE
AUX	—
Payload	status bytes in
Returns	'C'

Returns the drive's status — whether an image is mounted, and its read/write state.

Other Disk Commands

Cmd	Name	Purpose
50h	PUT	Write a sector without read-back verify
21h	FORMAT	Format the mounted image

4.4 The Clock (45h)

The clock device returns the date and time the FujiNet keeps from the internet. Several commands return the same moment in different formats; the most useful are below.

GET TIME — Read the time as bytes

Dev 45h Cmd 93h

Direction	Read
Field (DH)	NONE
AUX	—
Payload	6 bytes in
Returns	'C'

Returns the local date and time as six bytes: year (since 1900), month, day, hour, minute, second.

```
unsigned char t[6];
int_f5_read(0x45, 0x93, 0, 0, t, 6);
/* t[0]=year-1900 t[1]=mon t[2]=day t[3]=hr t[4]=min t[5]=sec */
```

GET TIME (ISO) — Read the time as text

Dev 45h Cmd 5Ah

Direction	Read
Field (DH)	NONE
AUX	—
Payload	ISO-8601 string in
Returns	'C'

Returns the time as a printable ISO-8601 string in UTC (for example 2026-06-13T18:20:08Z). Command 49h returns local time the same way.

SET TIMEZONE — Set the local time zone

Dev 45h Cmd 99h

Direction	Write
Field (DH)	NONE
AUX	—
Payload	TZ string out
Returns	'C'

Sets the POSIX time-zone string (for example **CST6CDT**) used by the local-time commands.

SECTION 5. A WORKED EXAMPLE: NETCAT

Nothing shows an interface better than a small, complete program. This section builds **NETCAT** — a bare-bones version of the classic network utility — entirely in 8088 assembly language, using nothing but **INT**

F5h for the network and **INT 21h** for the console. It opens a TCP connection, copies everything the connection sends to the screen, and sends everything you type back. In about forty instructions you have a working network terminal.

What It Does

NETCAT uses a single network adapter (device **71h**). After opening it on a TCP devicespec, the program loops:

- Ask the adapter how many bytes are waiting (STATUS).
- If the remote has closed, stop. If bytes are waiting, READ them and write them to standard output.
- If a key has been pressed, send it to the connection (WRITE). **ESC** ends the program.

Opening the Connection

A **.COM** program runs with all segment registers equal, so **ES** already points at our data once we copy **CS** into it. The devicespec is a plain string at the end of the program. We open for read/write (mode **0Ch**) with no translation, setting the field descriptor in **DH** to **A1_A2** so both AUX bytes are sent:

```
MOV AX,CS
MOV ES,AX          ; ES -> our data (for ES:BX)
MOV DX,0280h       ; DH=02 (A1_A2), DL=80 (write)
MOV AX,4F71h       ; AH=4Fh 'O' open, AL=71h adapter 1
MOV CL,0Ch         ; AUX1 = read/write
MOV CH,00h         ; AUX2 = no translation
MOV BX,OFFSET SPEC ; ES:BX -> devicespec string
MOV DI,SPECLEN     ; DI = its length
INT 0F5h
CMP AL,'C'
JNE QUIT           ; 'E' -> could not connect
```

The Polling Loop

The heart of the program asks for status, then acts. STATUS returns four bytes: the count of bytes waiting (low, high), a connected flag, and an error code. We read it into a 4-byte buffer:

```

POLL:  MOV DX,0040h          ; DH=00, DL=40 (read)
        MOV AX,5371h         ; AH=53h 'S' status, AL=71h
        MOV BX,OFFSET STAT
        MOV DI,4
        INT 0F5h
        CMP BYTE PTR [STAT+2],0 ; connected?
        JE  DONE             ; remote closed -> finish
        MOV AX,WORD PTR [STAT] ; AX = bytes waiting
        OR  AX,AX
        JZ  KBD              ; none waiting -> check keyboard

```

Reading and Writing

When bytes are waiting we READ them (capped to our buffer) and hand them to DOS for display. Note the field descriptor **A1_A2** and the count in both **CX** and **DI**:

```

        CMP AX,128
        JBE RLEN
        MOV AX,128          ; cap to buffer size
RLEN:   MOV CX,AX            ; CL/CH = count
        MOV DI,AX           ; DI = count
        MOV DX,0240h        ; DH=02 (A1_A2), DL=40 (read)
        PUSH AX
        MOV AX,5271h         ; AH=52h 'R' read, AL=71h
        MOV BX,OFFSET BUF
        INT 0F5h
        POP CX              ; CX = bytes read
        MOV BX,1            ; stdout handle
        MOV DX,OFFSET BUF
        MOV AH,40h          ; DOS 'write file'
        INT 21h

```

A keypress is sent to the connection one byte at a time. DOS function **0Bh** tells us whether a key is ready; **0Bh** reads it without echo:

```

KBD:    MOV    AH,0Bh           ; key ready?
        INT    21h
        OR     AL,AL
        JZ     POLL            ; no -> keep polling
        MOV    AH,08h          ; read key, no echo
        INT    21h
        CMP    AL,27           ; ESC ends the program
        JE     DONE
        MOV    [BUF],AL
        MOV    DX,0280h        ; DH=02 (A1_A2), DL=80 (write)
        MOV    AX,5771h        ; AH=57h 'W' write, AL=71h
        MOV    CX,1            ; one byte
        MOV    BX,OFFSET BUF
        MOV    DI,1
        INT    0F5h
        JMP    POLL

```

The Whole Program

Closing the adapter and exiting completes it. Assemble with TASM (**TASM NETCAT** then **TLINK /t NETCAT**) or MASM (**ML /AT NETCAT.ASM**) to produce **NETCAT.COM**.

```
; NETCAT.ASM -- a tiny netcat for FujiNet, via INT F5h.
```

```
.MODEL TINY
.CODE
ORG 100h
START: MOV AX,CS
      MOV ES,AX
      MOV DX,0280h      ; open: A1_A2 / write
      MOV AX,4F71h      ; 'O', adapter 1
      MOV CL,0Ch        ; read/write
      MOV CH,00h        ; no translation
      MOV BX,OFFSET SPEC
      MOV DI,SPECLEN
      INT 0F5h
      CMP AL,'C'
      JNE QUIT
POLL:  MOV DX,0040h      ; status
      MOV AX,5371h
      MOV BX,OFFSET STAT
      MOV DI,4
      INT 0F5h
      CMP BYTE PTR [STAT+2],0
      JE DONE
      MOV AX,WORD PTR [STAT]
      OR AX,AX
      JZ KBD
      CMP AX,128
      JBE RLEN
      MOV AX,128
RLEN:  MOV CX,AX
      MOV DI,AX
      MOV DX,0240h      ; read: A1_A2 / read
      PUSH AX
      MOV AX,5271h
      MOV BX,OFFSET BUF
      INT 0F5h
      POP CX
      MOV BX,1
      MOV DX,OFFSET BUF
      MOV AH,40h
      INT 21h
KBD:   MOV AH,0Bh
      INT 21h
      OR AL,AL
      JZ POLL
      MOV AH,08h
      INT 21h
      CMP AL,27
      JE DONE
      MOV [BUF],AL
      MOV DX,0280h      ; write: A1_A2 / write
      MOV AX,5771h
      MOV CX,1
      MOV BX,OFFSET BUF
      MOV DI,1
      INT 0F5h
      JMP POLL
DONE:  MOV DX,0000h      ; close: none
```

Note: NETCAT polls as fast as it can, which is fine for a demonstration. A real program would yield to DOS between polls, or raise the byte-waiting interrupt rate (network command **5Ah**) and wait on it, to be gentler on the machine.

The Same Program in C

For comparison, here is NETCAT again — the same program, the same steps — in C, using the FujiNet library **fujinet-lib** instead of raw **INT**

F5h. Read the two side by side as a Rosetta stone: each library call below reduces to one of the interrupt calls on the facing pages. **network_open** is the OPEN of Section 4.2; **network_status** returns the same four bytes this program reads by hand; **network_read** and **network_write** are the READ and WRITE; **network_close** is the CLOSE. The library simply loads the registers and issues **INT F5h** for you (Section 2, *The C Wrappers*).

```

/* NETCAT.C -- the program above, in C, via fujinet-lib.
   Open Watcom: wcl -bcl=dos netcat.c fujinet.lib          */

#include <conio.h>          /* kbhit, getch          */
#include <stdio.h>          /* fwrite, printf        */
#include <fujinet-network.h>

#define SPEC "N:TCP://192.168.1.10:9000/"

int main(void)
{
    unsigned char  buf[128];
    unsigned short waiting; /* bytes ready to read    */
    unsigned char  connected; /* is the link still up? */
    unsigned char  err;      /* detailed error code     */
    int            n, key;

    network_init();
    if (network_open(SPEC, OPEN_MODE_RW, OPEN_TRANS_NONE) != FN_ERR_OK) {
        printf("could not connect\n");
        return 1;
    }

    for (;;) {
        /* STATUS: bytes waiting, and are we still connected? */
        network_status(SPEC, &waiting, &connected, &err);
        if (!connected)
            break; /* remote closed */

        if (waiting) { /* READ and print */
            if (waiting > sizeof(buf))
                waiting = sizeof(buf);
            n = network_read(SPEC, buf, waiting);
            if (n > 0)
                fwrite(buf, 1, n, stdout);
        }

        if (kbhit()) { /* a key? WRITE it */
            key = getch();
            if (key == 27) /* ESC quits */
                break;
            buf[0] = (unsigned char) key;
            network_write(SPEC, buf, 1);
        }
    }

    network_close(SPEC); /* CLOSE */
    return 0;
}

```

The C is shorter and clearer, and it is what most programs should use. The assembler version exists to show what **fujinet-lib** does underneath — and to prove that the whole interface fits in a handful of register loads and one interrupt.

SECTION 6. A SECOND EXAMPLE: READING MASTODON

Most of the modern web answers in **JSON**, and JSON is a poor fit for a machine with 64 kilobytes and no string library. The FujiNet solves this on your behalf: it can fetch a web page, parse the JSON *on the adapter*, and hand back only the field you ask for. Your program never sees a brace.

This example fetches the newest post from a public Mastodon timeline and pulls out three fields — the author's display name, the date, and the body of the post. It is the heart of the FujiNet Mastodon client, reduced to its network calls and shown three ways.

The JSON Engine

Three network commands do the work, after an ordinary OPEN:

- **CHANNEL MODE (Fch)** with **AUX1 = 01h** switches the adapter into **JSON mode**.
- **PARSE (50h, 'P')** tells the adapter to read the whole HTTP response and parse it as JSON, holding the result in the FujiNet.
- **QUERY (51h, 'Q')** sends a path such as **/0/content** a following **STATUS** gives the length of the matching value, and a **READ** returns it.

A path is a slash-separated walk into the JSON: **/0** is the first element of the top-level array (the newest post), **/0/account/display_name** the author's name within it. Query as many paths as you like before closing.

The Flow

- **network_open** the timeline URL for an HTTP GET.
- **network_json_parse** — set JSON mode, then PARSE.
- **network_json_query** once per field — QUERY, STATUS, READ.
- **network_close**.

The URL asks the server for just the newest post:

```
N:HTTPS://oldbytes.space/api/v1/timelines/public?limit=1
```

In C

With **fujinet-lib** the whole job is a dozen lines. This is essentially the client's own **fetch_post** routine:

```
/* MASTO.C -- newest Mastodon post, three fields, via fujinet-lib. */
#include <string.h>
```



```

#include <fujinet-network.h>

static const char *url =
    "N:HTTPS://oldbytes.space/api/v1/timelines/public?limit=1";

void fetch_post(char *name, char *date, char *toot)
{
    char tmp[256];

    memset(name, 0, 32);
    memset(date, 0, 32);
    memset(toot, 0, 5120);

    network_init();
    network_open(url, OPEN_MODE_HTTP_GET, 0x00);
    network_json_parse(url); /* JSON mode + PARSE */

    network_json_query(url, "/0/account/display_name", tmp);
    strncpy(name, tmp, 31);
    network_json_query(url, "/0/created_at", tmp);
    strncpy(date, tmp, 31);
    network_json_query(url, "/0/content", toot); /* QUERY+STATUS+READ */

    network_close(url);
}

```

In Assembler

The same sequence in 8088, calling **INT F5h** directly. A subroutine **QUERY** sends one path and reads its value, so the three fields are three calls. Note the JSON-mode and PARSE calls between OPEN and the first query.

```

; MASTO.ASM -- newest Mastodon post fields via INT F5h.
.MODEL SMALL
.STACK 100h
.DATA
URL    DB 'N:HTTPS://oldbytes.space/api/v1/timelines/public?limit=1',0
URLLEN EQU $-URL
QNAME  DB '/0/account/display_name',0
QDATE  DB '/0/created_at',0
QBODY  DB '/0/content',0
STAT   DB 4 DUP(0)
NAME   DB 32 DUP(0)
DATE_  DB 32 DUP(0)
BODY   DB 5120 DUP(0)      ; a toot can be long
.CODE
START: MOV AX,@DATA
       MOV DS,AX
       MOV ES,AX           ; ES=DS for ES:BX payloads

```

```

MOV DX,0080h      ; open: DH=00, DL=80 (write)
MOV AX,4F71h      ; 'O', adapter 1
MOV CL,04h        ; AUX1 = HTTP GET
MOV CH,00h        ; AUX2 = no translation
MOV BX,OFFSET URL
MOV DI,URLELEN
INT 0F5h
MOV DX,0000h      ; channel mode: none
MOV AX,0FC71h     ; AH=FCh
MOV CL,01h        ; AUX1 = JSON mode
INT 0F5h
MOV DX,0000h      ; parse: none
MOV AX,5071h      ; AH=50h 'P'
INT 0F5h
MOV SI,OFFSET QNAME ; query each field
MOV BP,OFFSET NAME
CALL QUERY
MOV SI,OFFSET QDATE
MOV BP,OFFSET DATE_
CALL QUERY
MOV SI,OFFSET QBODY
MOV BP,OFFSET BODY
CALL QUERY
MOV DX,0000h      ; close: none
MOV AX,4371h      ; 'C'
INT 0F5h
MOV AX,4C00h
INT 21h
; QUERY: path at DS:SI, destination at DS:BP
QUERY: MOV DX,0080h      ; write the path
MOV AX,5171h      ; AH=51h 'Q'
MOV BX,SI
MOV DI,256        ; the library sends 256
INT 0F5h
MOV DX,0040h      ; status -> bytes waiting
MOV AX,5371h      ; AH=53h 'S'
MOV BX,OFFSET STAT
MOV DI,4
INT 0F5h
MOV CX,WORD PTR [STAT] ; CL/CH = byte count
JCXZ Q9
MOV DX,0240h      ; read: DH=02 (A1_A2), DL=40
MOV AX,5271h      ; AH=52h 'R'
MOV BX,BP         ; ES:BX -> destination
MOV DI,CX         ; DI = count
INT 0F5h
Q9: RET
END START

```

In Pascal

And once more in Turbo Pascal, reaching **INT F5h** through the **Dos** unit's **Intr**. A small **FujiF5** helper plays the part the library's wrappers play in C; **JsonQuery** mirrors **network_json_query**.

```
{ MASTO.PAS -- newest Mastodon post via INT F5h (Turbo Pascal). }
uses Dos, Strings;

const
  DEV = $71;
  url  : PChar = 'N:HTTPS://oldbytes.space/api/v1/timelines/public'
            + '?limit=1';
  qName : PChar = '/0/account/display_name';
  qDate : PChar = '/0/created_at';
  qBody : PChar = '/0/content';
var
  stat : array[0..31] of Byte;

function FujiF5(dir, fld, cmd, aux1, aux2: Byte;
                buf: Pointer; len: Word): Char;
var r: Registers;
begin
  FillChar(r, SizeOf(r), 0);
  r.DL := dir;  r.DH := fld;
  r.AL := DEV;  r.AH := cmd;
  r.CL := aux1; r.CH := aux2;
  if buf <> nil then begin
    r.ES := Seg(buf^);  r.BX := OfS(buf^);  r.DI := len;
  end;
  Intr($F5, r);
  FujiF5 := Chr(r.AL);
end;

procedure JsonQuery(query, dest: PChar);
var bw: Word;
begin
  FujiF5($80, $00, $51, 0, 0, query, 256);      { 'Q' write path }
  FujiF5($40, $00, $53, 0, 0, @stat, 4);        { 'S' status   }
  bw := stat[0] or (Word(stat[1]) shl 8);
  if bw > 0 then
    FujiF5($40, $02, $52, Lo(bw), Hi(bw), dest, bw); { 'R' read   }
  dest[bw] := #0;
end;

procedure FetchPost(name, date, toot: PChar);
begin
  FillChar(name^, 32, 0);  FillChar(date^, 32, 0);
  FillChar(toot^, 5120, 0);
  FujiF5($80, $00, $4F, $04, 0, url, StrLen(url)+1); { open HTTP GET }
  FujiF5($00, $00, $FC, $01, 0, nil, 0);             { JSON mode   }
```

```

FujiF5($00, $00, $50, 0, 0, nil, 0):          { parse          }
JsonQuery(qName, name);
JsonQuery(qDate, date);
JsonQuery(qBody, toot);
FujiF5($00, $00, $43, 0, 0, nil, 0):          { close          }
end;

```

Note: All three speak to the same engine in the same order — open, JSON mode, parse, query each field, close. C names the steps, Pascal wraps them, and the assembler shows the registers underneath. The FujiNet does the parsing; the program just asks for ~~0~~**content** and is handed the text.

SECTION 7. INSIDE FUJINET.SYS

FUJINET.SYS is the program behind everything in this book. It is an installable MS-DOS **block device driver**, loaded from **CONFIG.SYS**, that does two jobs at once: it presents the FujiNet's eight virtual disks to DOS as ordinary drive letters, and — on the way — it installs the **INT F5h** handler documented in Sections 2 through 4. This section walks its source.

The DOS side of the story (how DOS finds, calls, and talks to a device driver) is covered in the *IBM DOS Technical Reference* here we follow what **FUJINET.SYS** does in answer.

The Device Header

Every driver begins with a **device header**: a fixed structure DOS reads to learn the driver's kind, its entry points, and its name. For **FUJINET.SYS** it is written in assembly, at offset 0 of the load image:

```
_SYS_HEADER segment word public 'SYS_HEADER'
    org      0
_sys_hdr_ label near
    extrn    Strategy_:near
    extrn    Interrupt_:near
ATTRIB_OVER_32M equ    0020h
ATTRIB_FAT_BPB  equ    2000h
ATTRIB_RW_IOCTL equ    4000h
    dd      -1          ; link to next driver (set by DOS)
    dw      ATTRIB_OVER_32M OR ATTRIB_FAT_BPB OR ATTRIB_RW_IOCTL
    dw      Strategy_    ; offset of STRATEGY entry
    dw      Interrupt_   ; offset of INTERRUPT entry
    db      8, 0, 0, 0, 0, 0, 0, 0 ; 8 units; rest unused
```

The fields, in order:

- **Link field** (**dd -1**) — a far pointer to the next driver in the chain. DOS fills this in when it links the driver; the initial -1 marks the end.
- **Attribute word** — bit flags describing the driver. A **zero** top bit marks it a **block** device (a character device sets bit 15). The three flags raised here say the driver understands volumes over 32 MB (**0020h**), supplies its own BPB so DOS accepts a non-IBM disk layout (**2000h**), and accepts IOCTL read/write control calls (**4000h**).
- **Strategy and Interrupt offsets** — the two entry points DOS calls (next).
- **Name/unit field** — eight bytes. For a block driver the first byte is the **unit count**: **8**, the FujiNet's eight disks. (A character driver puts an eight-character device name here instead — see **FUJIPRN.SYS**.)

Strategy and Interrupt

DOS calls a driver in **two steps**. First it calls the **strategy** routine with a far pointer (in **ES:BX**) to a **request packet** the strategy routine only stows that pointer away. Then it calls the **interrupt** routine, which does the actual work on the saved packet. The split is a relic of multitasking ambitions that DOS never used, but every driver must honor it. In **FUJINET.SYS** the two are C functions with custom calling conventions. Strategy is a single line:

```
static SYSREQ __far *fpRequest = (SYSREQ __far *) 0;

void far Strategy(SYSREQ far *req)
#pragma aux Strategy __parm [_es __bx]    // packet arrives in ES:BX
{
    fpRequest = req;                      // just remember it
    return;
}
```

The interrupt routine saves every register, switches to the driver's own stack, dispatches the command, marks the packet done, restores the stack and registers, and returns:

```
void far Interrupt(void)
{
    push_regs();

    // Save DOS's SS:SP and switch to our own 1 KB stack
    // (DOS calls drivers on a tiny stack).
    _asm {
        mov dos_ss, ss
        mov dos_sp, sp
        mov ss, ax          // ax = our_ss, cx = our_sp
        mov sp, cx
    }

    if (fpRequest->command > MAXCOMMAND
        || !(currentFunction = dispatchTable[fpRequest->command]))
        fpRequest->status = ERROR_BIT | UNKNOWN_CMD;
    else
        fpRequest->status = currentFunction(fpRequest);

    fpRequest->status |= DONE_BIT;          // tell DOS we finished

    _asm {
        // restore DOS's stack
        mov ss, dos_ss
        mov sp, dos_sp
    }
    pop_regs();
}
```

Note: The stack swap matters. DOS hands a driver only a few dozen bytes of stack — far too little for C code that calls into the serial routines. **FUJINET.SYS** moves to its own 1 KB stack for the duration of the call, then puts DOS's stack back exactly as it was.

The Request Packet

DOS describes each operation in a **request packet**. The first five bytes are common to every command — a length, the target unit, the command code, and a status word DOS reads on return — followed by a union of command-specific fields:

```
typedef struct {
    uint8_t length;           // size of this packet
    uint8_t unit;             // which disk (0-7)
    uint8_t command;          // what to do (0-24)
    uint16_t status;          // driver writes the result here
    uint8_t reserved[8];
    union {                   // one arm per command group:
        SYSREQ_INIT init;    // command 0 (INIT)
        SYSREQ_MEDIA media;  // command 1 (MEDIA CHECK)
        SYSREQ_BPB bpb;      // command 2 (BUILD BPB)
        SYSREQ_IO io;         // commands 4 & 8 (READ / WRITE)
        SYSREQ_INPUT input;
        SYSREQ_LDMAP ldmap;
    };
} SYSREQ;
```

The **command** byte is the whole story: it selects both the arm of the union that is valid and the handler that runs. The status word the driver writes back carries an **error bit** (**8000h**), a **done bit** (**0100h**), and, on error, a DOS error number in the low byte.

The Dispatch Table

The command byte (0 through 24) indexes a table of handlers. Commands the FujiNet has no use for point at **Unknown_cmd**, which reports *unknown command* to DOS:

```
static driverFunction_t dispatchTable[] = {
    Init_cmd,           // 0  INIT           (install-time)
    Media_check_cmd,    // 1  MEDIA CHECK    (has the disk changed?)
    Build_bpb_cmd,      // 2  BUILD BPB      (describe the disk)
    Ioctl_input_cmd,    // 3  IOCTL READ     (the FUJI signature)
    Input_cmd,          // 4  READ           (sectors in)
    Input_no_wait_cmd,  // 5  (char only)
    Input_status_cmd,   // 6  (char only)
    Input_flush_cmd,    // 7  (char only)
    Output_cmd,         // 8  WRITE          (sectors out)
    Output_verify_cmd,  // 9  WRITE+VERIFY
    Output_status_cmd,  // 10 (char only)
    Output_flush_cmd,   // 11 (char only)
```

```

Ioctl_output_cmd,    // 12  IOCTL WRITE
Dev_open_cmd,       // 13  OPEN
Dev_close_cmd,      // 14  CLOSE
Remove_media_cmd,   // 15  REMOVE MEDIA
Unknown_cmd, Unknown_cmd, Unknown_cmd, // 16-18 reserved
Ioctl_cmd,          // 19  GENERIC IOCTL
Unknown_cmd, Unknown_cmd, Unknown_cmd, // 20-22 reserved
Get_l_d_map_cmd,    // 23  GET LOGICAL DRIVE MAP
Set_l_d_map_cmd     // 24  SET LOGICAL DRIVE MAP
};

```

Of these, only a handful do real work: **INIT**, **MEDIA CHECK**, **BUILD BPB**, **IOCTL READ**, **READ**, and **WRITE**. The rest are answered politely and otherwise ignored.

Initialization

Command 0, **INIT**, runs once, when DOS loads the driver at boot. It is the busiest handler. Abridged, it:

```

uint16_t Init_cmd(SYSREQ far *req)
{
    // 1. Announce ourselves (DOS version via INT 21h AH=30h).
    consolef("\nFujiNet driver " VERSION " ... on MS-DOS %i.%i\n", ...);

    // 2. Turn the CONFIG.SYS line into environment variables
    //     (FUJI_PORT=, FUJI_BPS=, NOTIME) for getenv().
    parse_config(req->init.bpb_ptr);
    environ = (char **) &config_env;

    // 3. Open the serial port and identify the UART.
    fujicom_init();
    check_uart();

    // 4. Prove the FujiNet is really there; bail out if not.
    err = get_fujinet_version();
    if (!err) err = get_set_time(!getenv("NOTIME"));
    if (err) {
        // nothing answered:
        req->init.num_units = 0;           // claim zero drives,
        req->init.end_ptr   = 0;           // keep no memory resident
        return ERROR_BIT;
    }

    // 5. Hand DOS a BPB for each of the 8 disks (see below).
    req->init.num_units = FN_MAX_DEV;
    // ... fill fn_bpb_table[idx], point fn_bpb_pointers[idx] at it ...
    req->bpb.table = MK_FP(getCS(), fn_bpb_pointers);

    // 6. Report which drive letters we landed on, then arm INT F5.
    find_drive_letter(req->init.num_units);
}

```



```

    setf5();
    return OP_COMPLETE;
}

```

A few details repay a closer look.

The BIOS Parameter Block

Step 5 gives DOS a **BPB** — a BIOS Parameter Block — describing each disk's geometry. Rather than ask the FujiNet, the driver simply describes a familiar **360 KB 5.25-inch floppy**: 512-byte sectors, two sectors per cluster, two FATs, 112 root entries, **0x2d0** (720) sectors, media descriptor **0xFD**. Whatever image is later mounted, DOS reads its real boot sector through BUILD BPB and adjusts.

Finding the Drive Letter

Step 6's `find_drive_letter` performs a small piece of DOS archaeology to report the letters the FujiNet's disks received. It asks for the **List of Lists** (the undocumented internal table at the head of DOS) and reads the running block-device count from it:

```

void find_drive_letter(uint8_t num_units)
{
    uint8_t far *lol;
    _asm {
        mov ah, 52h          // INT 21h AH=52h: get List of Lists
        int 21h              // -> ES:BX
        mov word ptr lol,    bx
        mov word ptr lol+2, es
    }
    // byte at offset 0x20 = number of block devices so far,
    // which is also the index of our first drive letter.
    char first = lol[0x20] + 'A';
    consolef("FujiNet attached to drives %c:-%c:\n",
             first, first + num_units - 1);
}

```

Reading and Writing Sectors

The two commands DOS leans on hardest are **READ** (4) and **WRITE** (8). Both carry a **SYSREQ_IO** arm: a buffer, a sector count, and a starting sector. The driver loops, turning each DOS sector into a disk-device command on the FujiBus.

READ translates almost directly to the disk-device READ of Section 4.3 — the 32-bit sector goes into AUX1–AUX4 with the **C1234** field descriptor, and 512 bytes come back:

```

uint16_t Input_cmd(SYSREQ far *req)
{
    uint8_t far *buf = req->io.buffer_ptr;
    uint32_t sector = (req->length > 22) ? req->io.start_sector_32

```

```

                                : req->io.start_sector;
uint32_t sector_max = fn_bpb_table[req->unit].num_sectors;

for (idx = 0; idx < req->io.count; idx++, sector++) {
    if (sector >= sector_max)        // off the end of the image?
        return ERROR_BIT | NOT_FOUND;
    if (!fuji_bus_call(FUJI_DEVICEID_DISK + req->unit,
                      FUJICMD_READ, FUJI_FIELD_C1234,
                      U16_LSB(U32_LSW(sector)), U16_MSB(U32_LSW(sector)),
                      U16_LSB(U32_MSW(sector)), U16_MSB(U32_MSW(sector)),
                      NULL, 0, &buf[idx * SECTOR_SIZE], SECTOR_SIZE))
        break;                      // serial error: stop here
    }
    req->io.count = idx;              // tell DOS how many we did
    return idx ? OP_COMPLETE : (ERROR_BIT | GENERAL_FAIL);
}

```

The `U16_LSB(U32_LSW(...))` nest just slices the 32-bit sector into four bytes for AUX1–AUX4. On any serial failure the loop stops and reports the partial count, exactly as DOS expects of real hardware.

WRITE is the mirror image, with one guard in front: it refuses a disk that was mounted read-only, returning DOS's *write protect* error so the familiar *Write protect error writing drive X* appears:

```

uint16_t Output_cmd(SYSREQ far *req)
{
    if (disk_slots[req->unit].mode != SLOT_READWRITE)
        return ERROR_BIT | WRITE_PROTECT;    // read-only image
    // ... identical loop, FUJICMD_WRITE, buffer -> FujiNet ...
}

```

Media Check and BPB

Two smaller commands keep DOS's idea of the disk honest.

MEDIA CHECK (1) answers a single question: has the disk changed since DOS last looked? The driver asks the FujiNet for the slot's **mount time** and compares it with the value from before. If they differ, a new image has been mounted, and the driver tells DOS to throw away its cached directory and FAT:

```

fuji_bus_call(FUJI_DEVICEID_FUJINET, FUJICMD_STATUS, FUJI_FIELD_A1,
              STATUS_MOUNT_TIME, 0, 0, 0, NULL, 0,
              mount_status, sizeof(mount_status));
new_status = mount_status[req->unit * 2];
if (!new_status)      req->media.return_info = 0; // unsure
else if (old != new_status) req->media.return_info = -1; // changed
else                  req->media.return_info = 1; // unchanged

```

This is what lets you mount a different disk image in CONFIG and have it appear under the same drive letter without rebooting.

BUILD BPB (2) hands DOS the disk's real geometry. The driver reads sector 0 — the boot sector — of the freshly mounted image and copies the BPB out of it, from the standard offset **0x0B**:

```
fuji_bus_call(FUJI_DEVICEID_DISK + req->unit, FUJICMD_READ,
              FUJI_FIELD_C1234, 0, 0, 0, 0, NULL, 0, buf, SECTOR_SIZE);
_fmemcpy(fn_bpb_pointers[req->unit], &buf[0x0b], sizeof(DOS_BPB));
```

The IOCTL Signature

How does a utility such as **FMOUNT** know which drive letters belong to the FujiNet? Through **IOCTL READ** (command 3). A program opens the drive, issues a control read, and the driver answers with a four-byte signature and the unit number:

```
uint16_t ioctl_input_cmd(SYSREQ far *req)
{
    fuji_ioctl_query far *query = (fuji_ioctl_query far *) req->io.buffer_ptr;
    _fmemcpy(query->signature, "FUJI", 4); // the secret handshake
    query->unit = req->unit;
    return OP_COMPLETE;
}
```

Any program that reads **FUJI** back from a drive knows it is talking to a FujiNet, and which of the eight units it is. That is the whole mechanism by which the command-line tools find their device.

SECTION 8. INSIDE FUJIPRN.SYS

The companion driver, **FUJIPRN.SYS**, makes the FujiNet's printer device (ID **40h**) appear to DOS as the printer **LPT1**. Everything a program prints — through DOS or through the BIOS — is captured, buffered, and forwarded to the FujiNet, which renders it (to a PDF, for instance). It is a smaller driver than **FUJINET.SYS**, but it shows two techniques worth study: hooking the BIOS printer interrupt, and flushing safely from idle time.

A Character Device

Where **FUJINET.SYS** is a block driver, **FUJIPRN.SYS** is a **character** driver — a single stream, not a set of disks. Its header differs in just two places: the attribute word sets bit 15 (**8000h**) to mark a character device, and the name/unit field carries an eight-character **device name** rather than a unit count:

```
_SYS_HEADER segment word public 'SYS_HEADER'
    org      0
_sys_hdr_ label near
    extrn    Strategy_:near
    extrn    Interrupt_:near
    dd      -1                      ; link to next driver
    dw      8000h                  ; bit 15 = character device
    dw      Strategy_
    dw      Interrupt_
    db      'L','P','T','1',' ',' ',' ',' ',' ' ; device name
```

Because it names itself **LPT1**, DOS routes the printer stream to it. The strategy and interrupt routines, the request packet, and the dispatch table are the same machinery as in **FUJINET.SYS** (Section 7); only the handlers differ. Most are no-ops — a character printer has nothing to say about media or sectors — so this section follows only the few that matter.

Capturing Output

When DOS writes to the printer it calls **OUTPUT** (command 8), one byte at a time. The handler simply feeds the byte to the buffer:

```
uint16_t Output_cmd(SYSREQ far *req)
{
    if (!prn_buf_add(req->io.buffer_ptr[0]))
        return ERROR_BIT | NOT_READY;
    return OP_COMPLETE;
}
```

CLOSE (14) and **OUTPUT FLUSH** (11) push whatever is buffered to the FujiNet at once; **OUTPUT STATUS** (10) always answers *ready*. Everything else returns *unknown command*.

The INT 17h Hook

DOS is not the only way to print. Many programs call the BIOS printer service, **INT 17h**, directly — bypassing the driver entirely. To catch them too, **FUJIPRN.SYS** installs its own **INT 17h** handler at INIT time and answers the three BIOS printer functions itself:

```
int int17(uint16_t direction, uint16_t cmdchar, ...)
{
    unsigned char ah = cmdchar >> 8;    // BIOS function in AH
    unsigned char al = cmdchar & 0xFF;    // character in AL
    switch (ah) {
        case 0: prn_buf_add(al); return 0x9000; // print AL; status = ready
        case 1: return 0x9000; // init printer
        case 2: return 0x9000; // read status
    }
    return 0;
}
```

Each returns **90h** in **AH** — the BIOS status bits for *selected, acknowledged, not busy* — so callers believe a real, ready printer answered. Function 0 also buffers the character, exactly as the DOS path does, so the two routes converge on the same buffer.

The handler is reached through a small assembly wrapper that preserves registers, sets **DS** to the driver's segment, calls the C function, and returns with **IRET**:

```
INTERRUPT MACRO func                ; build an interrupt wrapper
    push    bx
    ... push cx dx si di bp ds es ...
    push    cs
    pop     ds                      ; DS = CS (our data segment)
    call    func
    ... pop es ds bp di si dx cx bx ...
    iret
ENDM

INTERRUPT int17_
```

The Buffer

Sending one byte per network packet would be painfully slow, so the driver gathers bytes in a 255-byte buffer and forwards them in bursts. A burst is a single FujiBus WRITE to the printer device:

```

int flush_prn_buf(void)
{
    int ok = 1;
    if (prn_buf_len > 0) {
        ok = fujiF5_write(FUJI_DEVICEID_PRINTER, FUJICMD_WRITE,
                        FUJI_FIELD_NONE, 0, 0, prn_buf, prn_buf_len);
        if (ok) { prn_buf_len = 0; memset(prn_buf, 0, PRN_BUF_SIZE); }
    }
    return ok;
}

```

A byte is added, and the buffer flushed when it nears full:

```

int prn_buf_add(unsigned char c)
{
    if (prn_buf_len >= PRN_BUF_SIZE)        // full: flush first
        if (!flush_prn_buf()) return 0;
    prn_buf[prn_buf_len++] = c;
    if (!buffering_enabled)                 // DOS < 3.0: flush every byte
        return flush_prn_buf();
    last_activity = timer_counter;          // note the time, for auto-flush
    if (prn_buf_len >= PRN_BUF_FLUSH)       // 250 of 255: flush
        flush_prn_buf();
    return 1;
}

```

But a document rarely ends on a buffer boundary. The last few bytes of a print job would sit in the buffer forever unless something flushed them — which is what the timer is for.

Auto-Flush: Timer and Idle

The driver flushes the tail of a job automatically, a few seconds after the printing stops. Doing so safely is the subtle part, because a flush calls into DOS to talk to the serial port, and **DOS is not re-entrant** — calling it from a hardware interrupt while it is already busy will corrupt it.

The solution uses two interrupts and one flag. At INIT the driver hooks the **timer tick** (**INT 08h**, 18.2 times a second) and the **DOS idle** interrupt (**INT 28h**), and records the address of DOS's **InDOS flag** — a byte DOS keeps non-zero while it is inside itself:

```

void install_timer_handler(void)
{
    _asm { mov ah, 34h    int 21h    // INT 21h AH=34h: address of
        mov indos_off, bx mov indos_seg, es } // the InDOS flag
    indos_flag = MK_FP(indos_seg, indos_off);

    // chain onto INT 08h (timer) and INT 28h (DOS idle)
    old = _dos_getvect(0x08); ...; _dos_setvect(0x08, timer_vect);
}

```

```

old = _dos_getvect(0x2B); ...; _dos_setvect(0x2B, idle_vect);

_asm { mov ah, 30h int 21h mov buffering_enabled, al } // DOS ver
buffering_enabled = (buffering_enabled >= 3) ? 1 : 0;    // need 3.0+
}

```

Every timer tick, the driver checks whether the buffer has been idle for five seconds. If so — and only if DOS is *not* currently busy — it flushes. If DOS *is* busy, it cannot flush now, so it sets a flag and waits for the idle interrupt:

```

void timer_tick(void)          // INT 08h, 18.2 Hz
{
    timer_counter++;
    if (prn_buf_len > 0 && (timer_counter - last_activity) >= AUTO_FLUSH_TICKS) {
        if (indos_flag && *indos_flag == 0) { // safe: DOS is idle
            flush_prn_buf();
            last_activity = timer_counter;
        } else {
            flush_pending = 1;          // DOS busy -- let INT 28h do it
        }
    }
}

void idle_flush(void)          // INT 28h, raised by DOS when idle
{
    flush_pending = 0;
    flush_prn_buf();           // here it is always safe to call DOS
}

```

DOS raises **INT 28h** precisely when it is sitting idle and willing to be re-entered — the one safe moment for a background task to call it. By deferring to that moment whenever it finds DOS busy, the driver flushes the tail of every print job without ever risking the system. The timer wrappers are ordinary interrupt stubs that set **DS**, call the C routine, and **chain** to the previous handler rather than returning:

```

timer_vect_ PROC NEAR
    ... push registers; push cs; pop ds ...
    call timer_tick_
    ... pop registers ...
    jmp     dword ptr cs:[_old_timer_off] ; chain to old INT 08h
timer_vect_ ENDP

```

Chaining matters: the system clock and every other timer client must still see the tick. The driver does its small work and passes the interrupt along.

SECTION 9. APPENDICES

Appendix A. Network Error Codes

When a network command returns 'E', the detailed reason is the fourth byte of the next STATUS reply (Section 4.2). The values:

Code	Meaning	Code	Meaning
1	Success	167	Access denied
131	Write-only	170	File not found
132	Invalid command	200	Connection refused
135	Read-only	201	Network unreachable
136	End of file	202	Socket timeout
138	General timeout	203	Network down
144	General error	204	Connection reset
146	Not implemented	207	Not connected
151	File exists	208	Server not running
162	No space on device	210	Service unavailable
165	Invalid devicespec	212	Bad username/password

Appendix B. Status Request Types

The control device's STATUS command (70h, 53h) and the network adapters' local STATUS take a request type in AUX1, selecting which value to return:

AUX1	Returns
0	Connection / error status
1	IP address — or, for the control device, mount time
2	Netmask
3	Gateway
4	DNS server

Appendix C. Field Descriptors

The value placed in DH, and the number of AUX bytes it sends into the command frame (repeated from Section 2 for reference):

Name	DH	Bytes	Typical use
FUJI_FIELD_NONE	00h	0	no parameters
FUJI_FIELD_A1	01h	1	one slot or index
FUJI_FIELD_A1_A2	02h	2	two bytes, or a 16-bit length
FUJI_FIELD_A1_A2_A3	03h	3	three parameters
FUJI_FIELD_A1_A2_A3_A4	04h	4	four parameters
FUJI_FIELD_C1234	07h	4	a 32-bit value (disk sector)

Appendix D. The FujiBus Frame

For reference, the packet the driver builds and sends. All multi-byte fields are little-endian. The frame is SLIP-encoded on the wire.

Field	Bytes	Meaning
device	1	Destination device ID
command	1	Command code
length	2	Total packet length, including this header
checksum	1	Fold-add checksum of the whole packet
fields	1	Field descriptor (how many AUX bytes follow)
AUX	0–4	The AUX parameter bytes
payload	n	Command data, if any

The reply carries the same header with the command byte set to **ACK (06h)** on success, followed by any data the command returns. The driver checks the length, checksum, and device before reporting 'C'.



github.com/FujiNetWIFI · fujinet.online